



Reinforcement learning-based model predictive control for greenhouse climate control

Samuel Mallick^{a,*,1}, Filippo Airdi^{a,1}, Azita Dabiri^{a,1}, Congcong Sun^{b,1},
Bart De Schutter^{a,1}

^a Delft Center for Systems and Control, Delft University of Technology, 2628 CD, Delft, the Netherlands

^b Agricultural Biosystems Engineering Group, Wageningen University, 6700 AA, Wageningen, the Netherlands

ARTICLE INFO

Keywords:

Greenhouse climate control
Model predictive control
Reinforcement learning

ABSTRACT

Greenhouse climate control is concerned with maximizing performance in terms of crop yield and resource efficiency. One promising approach is model predictive control (MPC), which leverages a model of the system to optimize the control inputs, while enforcing physical constraints. However, prediction models for greenhouse systems are inherently inaccurate due to the complexity of the real system and the uncertainty in predicted weather profiles. For model-based control approaches such as MPC, this can degrade performance and lead to constraint violations. Existing approaches address uncertainty in the prediction model with robust or stochastic MPC methodology; however, these necessarily reduce crop yield due to conservatism and often bear higher computational loads. In contrast, learning-based control approaches, such as reinforcement learning (RL), can handle uncertainty naturally by leveraging data to improve performance. This work proposes an MPC-based RL control framework to optimize the climate control performance in the presence of prediction uncertainty. The approach employs a parametrized MPC scheme that learns directly from data, in an online fashion, the parametrization of the constraints, prediction model, and optimization cost that minimizes constraint violations and maximizes climate control performance. Simulations show that the approach can learn an MPC controller that significantly outperforms the current state-of-the-art in terms of constraint violations and efficient crop growth.

1. Introduction

Greenhouse climate control presents a key opportunity to address the growing world population's food production requirements in a changing climate, and the grand societal challenge of efficient energy consumption. With modern greenhouses equipped with actuation systems such as heating, ventilation, and CO₂ injection, effective control approaches can lead to high crop yield in an energy-efficient manner. However, the control challenge is difficult as the process dynamics are highly nonlinear and complex [28], and the climate variables, such as temperature and humidity, must be effectively constrained to avoid damage to crops due to, e.g., spread of diseases [6,34,11]. While traditional methods, e.g., on-off and PID control, have been used for low-level regulation, these strategies are not rooted in optimal control and are in general unable to deliver optimal performance and to systematically handle complex state and/or input constraints [12,15].

Model predictive control (MPC) is an optimization-based control methodology that naturally handles multi-input-multi-output systems with state, input, and output constraints [5]. It has seen huge theoretical success and application in process control, and has been proposed to solve greenhouse control challenges [3,10,21]. However, MPC relies heavily on an accurate prediction model, while in a greenhouse prediction model there always exist uncertainties due to, e.g., modeling error and inaccurate weather forecast. For model-based control, an incorrect model can lead the controller to drive the system to an undesired point of operation, possibly violating constraints. Furthermore, as constraints often represent the validity range for the accuracy of the prediction model, violations of the constraints can lead to degraded performance when applied to the real system [34].

Some existing works have addressed prediction uncertainty only in the context of external disturbances, such as weather predictions [6,14]. In [8], the uncertainty in market prices is addressed with a scenario-

* Corresponding author.

E-mail address: s.h.mallick@tudelft.nl (S. Mallick).

¹ These authors contributed equally to this work.

based stochastic MPC controller; however, the controller is based on mixed-integer optimization, requiring significant computational efforts. Alternatively, the following existing works have explored uncertainty stemming from an incorrect physical model of the system. In [12], the robustness of predictive control for a greenhouse in the presence of model uncertainties is considered empirically, however no mechanism to compensate for the prediction error is introduced. In [19,20], a neural network is used as a prediction model, with a robust MPC controller addressing the prediction uncertainty; however, a min-max robust MPC approach is used, which is inherently conservative. In [34], an approach to mitigating the negative effects of model uncertainty is presented using online parameter estimation. However, only a small subset of the (possibly) uncertain model parameters are estimated. Of particular note, in [4], parametric uncertainty in all model parameters is addressed. A robust sample-based MPC controller is proposed to incorporate the uncertainty into the control approach. However, the resulting control scheme is unable to adequately reduce constraint violations, and results in less crop yield due to conservativeness. More recently, [27] has proposed a chance-constrained stochastic MPC formulation to address parametric uncertainties in greenhouse production systems. However, this approach relies on linearization of the prediction model, further increasing prediction uncertainty. Additionally, the chance constrained formulation leads to a computational load that is higher than traditional MPC schemes.

In contrast to MPC, reinforcement learning (RL) is a model-free control methodology where a control policy is learned from data observed from the system [26]. RL controllers naturally handle uncertainty and adapt to changing conditions with no additional mechanisms, as they are learned through direct interaction with the real system. For complex systems with large continuous state and action spaces, the state-of-the-art for RL uses deep neural networks (DNNs) as function approximators to represent the controller. With the power of DNNs as general function approximators, this idea has seen unprecedented success on previously unsolved problems, e.g., the games of chess and Go [25]. The power and inherent uncertainty handling of RL has been identified as useful for greenhouse climate control, with [32] proposing a DNN-based RL approach based on deep deterministic policy gradient (DDPG) [17], and [22] drawing a comparison between MPC and DDPG. However, an inherent drawback for RL controllers is the absence of theoretical guarantees on the satisfaction of constraints and a lack of interpretability due to the black-box DNN function approximation. In the context of greenhouse control, this means growers have no guarantee that the automated controller will effectively constrain climate variables to safe ranges, with potential negative implications on crop health and profit.

Recently, [9] proposed and justified an integrated MPC and RL control paradigm, where the MPC controller serves as a function approximator for the optimal policy in model-based RL. Fig. 1 depicts a schematic overview of this architecture. In such a scheme, the MPC controller's optimization problem acts both as policy provider, picking actions based on a state, and value function approximator, estimating 'how good' it is to be in a given state. The learning algorithm, e.g., Q-learning [33], is tasked with adjusting the parametrization of the MPC controller in an effort to discover the optimal control policy, thus improving closed-loop performance in a data-driven fashion. In this way, despite the presence of mismatches between the prediction model and the real system, the MPC control scheme is able to learn and deliver, at convergence, the optimal policy and value functions of the underlying RL task, granted the MPC parametrization is rich enough. In contrast to DNN-based RL, the MPC scheme at the core of this approach provides the option of integrating prior information that may be known on the system in the form of, e.g., an expert-based, possibly imperfect, prediction model. Moreover, MPC-based agents are in general more amenable to analysis and certification in terms of stability and constraint satisfaction [5]. Finally, MPC-based controllers can take constraints into account in an explicit and structured way, which DNNs are generally incapable of doing. The above benefits render this methodology suitable for greenhouse climate

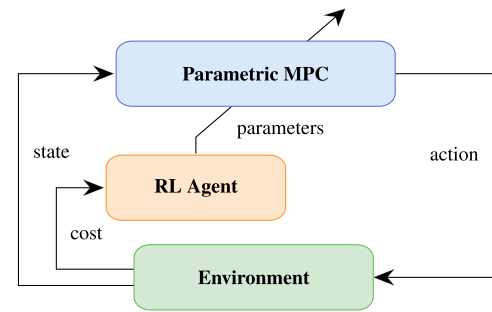


Fig. 1. Diagram of the MPC-based RL architecture.

control, where an inaccurate prediction model is known, and climate variables must be constrained.

Therefore, in this work we propose an integrated MPC and RL framework to address the problem of greenhouse climate control under parametric uncertainty stemming from uncertain weather predictions and modeling mismatches. Specifically:

1. To the best of the authors' knowledge, a combined MPC and RL approach for greenhouse climate control in the presence of uncertainty is proposed for the first time. A parametrized MPC scheme, inspired by [9], is crafted to serve as policy provider and value function approximator in an RL formulation of the greenhouse climate control problem. A second-order Q-learning algorithm is leveraged to adjust the parametrization of the MPC scheme online, automatically learning a control policy. The approach provides an adaptive and high-performing climate controller that minimizes potentially dangerous constraint violations, without negatively affecting crop growth due to robustness conservatism, and without relying on an expensive-to-acquire accurate prediction model. Additionally, in contrast to DNN-based learning approaches, the behavior of the resulting controller can be interpreted by analyzing the learned parametrization of the constraints, prediction model, and cost function.
2. The approach is then validated in simulation, and compared against both model-based MPC and model-free RL state-of-the-art controllers. The results demonstrate the effectiveness of the approach, with the proposed methodology outperforming existing controllers from the literature in both constraint satisfaction and efficient crop growth.

Compared to the state-of-the-art MPC formulations [4,8,19,20,27], instead of addressing uncertainty in the parametrization in a robust or stochastic fashion, the proposed methodology adapts its policy via RL in order to improve closed-loop performance. This has the distinct advantage of yielding less conservative control schemes while retaining low computational complexity. In comparison with DNN-based approaches such as [22,32], our method integrates MPC as function approximator in the RL algorithm, fostering a model-based approach that is more suitable for learning high-performance, constraint-abiding policies.

The paper is structured as follows. In Section 2 relevant background is provided on the greenhouse model used, and on the combined MPC and RL paradigm from [9]. The problem we address is formally defined in Section 3. Section 4 presents the proposed methodology, which is then applied and assessed extensively in simulation in Section 5. Finally, conclusions and future work directions are given in Section 6.

2. Background

This section describes the greenhouse model considered in this work. Additionally, background theory on combining MPC and RL is provided, upon which the methodology proposed in this paper is built.

Table 1
Physical meaning of the state x , output y , input u , and disturbance d .

x_1	dry-weight (kg m^{-2})	y_1	dry-weight (g m^{-2})	d_1	radiation (W m^{-2})
x_2	indoor CO_2 (kg m^{-3})	y_2	indoor CO_2 (%)	d_2	outdoor CO_2 (kg m^{-3})
x_3	indoor temperature ($^\circ\text{C}$)	y_3	indoor temperature ($^\circ\text{C}$)	d_3	outdoor temperature ($^\circ\text{C}$)
x_4	indoor humidity (kg m^{-3})	y_4	indoor humidity (%)	d_4	outdoor humidity (kg m^{-3})
u_1	CO_2 injection ($\text{mg m}^{-2} \text{s}^{-1}$)	u_2	ventilation (mm s^{-1})	u_3	heating (W m^{-2})

2.1. Lettuce greenhouse model

We consider a greenhouse for lettuce growing, with the continuous-time model, presented in [28], given in the Appendix. While the continuous-time model is used in all simulations, for control purposes a discrete-time model is considered

$$\begin{aligned} x(k+1) &= f(x(k), u(k), d(k), p), \\ y(k) &= g(x(k), p), \end{aligned} \quad (1)$$

with $x \in \mathbb{R}^4$ the state, $u \in \mathbb{R}^3$ the control input, $d \in \mathbb{R}^4$ the weather disturbance, and $y \in \mathbb{R}^4$ the output. Furthermore, $p \in \mathbb{R}^{28}$ is a set of model parameters, and $k \in \mathbb{Z}^+$ is the discrete-time counter for discrete time steps of $\Delta t = 900$ s (15 minutes). The nonlinear functions f and g , and the model parameters p , are given in the Appendix. The physical meaning of the states, outputs, inputs, and disturbances is summarized in Table 1. Estimation is out the scope of this paper, and it is assumed, as in [4], that at each time step k a perfect estimate of the state $x(k)$ is available.

2.2. MPC as a function approximator in RL

Consider discrete-time system dynamics as a Markov Decision Process (MDP) [26] with continuous state $s \in \mathbb{R}^n$, continuous action $a \in \mathbb{R}^m$, and state transitions $s, a \rightarrow s_+$ with the underlying conditional probability density

$$\mathbb{P}[s_+ | s, a] : \mathbb{R}^n \times \mathbb{R}^m \times \mathbb{R}^n \rightarrow [0, 1]. \quad (2)$$

Consider a deterministic policy $\pi_\theta(s) : \mathbb{R}^n \rightarrow \mathbb{R}^m$ parametrized by $\theta \in \mathbb{R}^l$. Selecting actions based on this policy will cause the system to visit the MDP's states with a given distribution, denoted η_{π_θ} . The performance of such a policy is defined as [26]

$$J(\pi_\theta) = \mathbb{E}_{\eta_{\pi_\theta}} \left[\sum_{k=0}^{N_s} \gamma^k L(s_k, \pi_\theta(s_k)) \right], \quad (3)$$

where s_k is the state at time step k , $L(s, a) : \mathbb{R}^n \times \mathbb{R}^m \rightarrow \mathbb{R}$ the stage cost, $\gamma \in (0, 1]$ the discount factor, and N_s the number of time steps considered in a task. The RL task is then to find the optimal policy π_θ^* as

$$\pi_\theta^* = \arg \min_{\theta} J(\pi_\theta). \quad (4)$$

The familiar notions of state- and action-value functions [26] are defined respectively as

$$Q_\theta(s_k, a_k) = L(s_k, a_k) + \mathbb{E}_{\eta_{\pi_\theta}} \left[\sum_{\tau=k+1}^{N_s} \gamma^{\tau-k} L(s_\tau, \pi_\theta(s_\tau)) \right], \quad (5)$$

and $V_\theta(s_k) = Q_\theta(s_k, \pi_\theta(s_k))$. While DNNs are the most common choice for representing the policy and value functions [17], their black-box nature does not facilitate the injection of prior information, e.g., approximate prediction models, nor is it conducive to an interpretable policy and the addition of constraints. To account for these drawbacks, [9] proposed the use of an MPC scheme in place of a DNN.

Consider the following MPC problem approximating the value function, parametrized by θ , $V_\theta : \mathbb{R}^n \rightarrow \mathbb{R}$ as

$$\begin{aligned} V_\theta(s) &= \min_{\mathbf{x}, \mathbf{u}, \sigma} \lambda_\theta(x(0)) \\ &\quad + \sum_{k=0}^{N-1} \gamma^k \left(L_\theta(x(k), u(k)) + \omega^\top \sigma(k) \right) \\ &\quad + \gamma^N \left(V_\theta^f(x(N)) + w_f^\top \sigma(N) \right) \end{aligned} \quad (6a)$$

$$\text{s.t.} \quad x(0) = s, \quad (6b)$$

$$\text{for } k = 0, \dots, N-1$$

$$x(k+1) = f_\theta(x(k), u(k)), \quad (6c)$$

$$h_\theta(x(k), u(k)) \leq \sigma(k), \quad (6d)$$

$$\sigma(k) \geq 0, \quad (6e)$$

$$h_\theta^f(x(N)) \leq \sigma(N), \quad (6f)$$

$$\sigma(N) \geq 0, \quad (6g)$$

where slack variable $\sigma(k)$ softens the inequality constraint for time step k , and the vectors $\mathbf{x}$, $\mathbf{u}$ and σ respectively collect the states, actions, and slack variables over the horizon N . Problem (6) is solved numerically online to generate the value $V_\theta(s)$. In (6), λ_θ is an initial cost term, L_θ is the stage cost, and V_θ^f is a terminal cost approximation, all of which are parametrized by θ . Furthermore, f_θ is the model approximation, and h_θ, h_θ^f are inequality constraints. Lastly, w and w_f are the weights of the slack variable in the objective. Note that the formulation (6) is general, i.e., the dimension of θ , and how it enters into the respective functions in (6), is not made explicit. In Section 4 we will introduce a concrete realization, designed for the greenhouse climate control problem.

Given (6), the action value function Q_θ and the policy π_θ , satisfying the fundamental equalities of the Bellman equations [9], are defined as follows:

$$\begin{aligned} Q_\theta(s, a) &= \min_{\mathbf{x}, \mathbf{u}, \sigma} \quad (6a) \\ \text{s.t.} \quad & (6b) - (6g), \end{aligned} \quad (7)$$

$$u(0) = a,$$

$$\pi_\theta(s) = \arg \min_a Q_\theta(s, a). \quad (8)$$

Therefore, in RL terms, the parametric MPC scheme acts as policy provider for the learning agent, whose goal is to modify the parameters θ of the controller in order to minimize (4). Various forms of RL [26] exist that solve this problem directly or indirectly via iterative updates

$$\theta \leftarrow \theta - \alpha \nabla_\theta \sum_{i=1}^m \psi(s_i, a_i, s_{i+1}, \theta), \quad (9)$$

where $\alpha \in \mathbb{R}_+$ is the learning rate, z denotes the number of observations used in the update (i.e., a batch of observations), and ψ captures the controller's performance and varies with the specific algorithm. For example, in recursive Q-learning we have that $m = 1$ and $\psi = \delta_i^2$, where the temporal-difference (TD) error

$$\delta_i = L(s_i, a_i) + \gamma V_\theta(s_{i+1}) - Q_\theta(s_i, a_i) \quad (10)$$

captures the estimation error of the value functions [26].

3. Problem formulation

We address the problem of optimal greenhouse climate control for crop yield and resource efficiency. As in [4], we consider the predictive uncertainty, stemming from uncertain weather predictions and modeling errors, to be captured by parametric uncertainty² in the model parameters p . Specifically, the true values for p are assumed to be unknown.

Growth cycles of 40 days are considered, where control inputs are computed at 15 minute time steps, i.e., growth cycles of $N_s = 3840$ time steps. This duration is in line with the literature [4,22], and represents a standard lettuce growth cycle, at the end of which the lettuce is harvested and sold, generating economic profit. During each growth cycle, we wish to maximize the yield and minimize the violations of constraints on the system outputs, whilst minimizing the cost associated with control signals. The system outputs are constrained as $y_{\min}(k) \leq y(k) \leq y_{\max}(k)$, with [4,22]

$$\begin{aligned} y_{\min}(k) &= (0, 0, y_{3,\min}(d_1(k)), 0)^\top, \\ y_{\max}(k) &= (\infty, 1.6, y_{3,\max}(d_1(k)), 70)^\top, \end{aligned} \quad (11)$$

and with the time-varying third element defined as

$$\begin{aligned} y_{3,\min}(d) &= \begin{cases} 10, & \text{if } d < 10 \\ 15, & \text{if } d \geq 10 \end{cases}, \\ y_{3,\max}(d) &= \begin{cases} 15, & \text{if } d < 10 \\ 20, & \text{if } d \geq 10 \end{cases}. \end{aligned} \quad (12)$$

These time varying constraints are common in the literature; they reflect that inside the greenhouse it is colder during the night than during the day [22,4,24]. The inputs are constrained as $u_{\min} \leq u(k) \leq u_{\max}$, with [4,22]

$$\begin{aligned} u_{\min} &= (0, 0, 0)^\top, \\ u_{\max} &= (1.2, 7.5, 150)^\top. \end{aligned} \quad (13)$$

Finally, the input rate is constrained as [4,22]

$$|u(k+1) - u(k)| \leq (1/10)u_{\max}. \quad (14)$$

Define the following performance indicators:

- Final yield $y_1(N_s)$
- Constraint violations Ψ
- Economic profit indicator P .

The final yield $y_1(N_s)$ is the dry lettuce weight at the end of a growth cycle of N_s time steps. The constraint violations indicator Ψ is defined as

$$\begin{aligned} \Psi = \sum_{k=0}^{N_s} \sum_{i=1}^4 \left(\max \left(0, \frac{y_i(k) - y_{i,\max}(k)}{y_{i,\max}(k) - y_{i,\min}(k)} \right) \right. \\ \left. + \max \left(0, \frac{y_{i,\min}(k) - y_i(k)}{y_{i,\max}(k) - y_{i,\min}(k)} \right) \right). \end{aligned} \quad (15)$$

² The choice to represent modeling error and weather prediction inaccuracy via the uncertainty in p results in a prediction model in which even the parameters that may represent known physical quantities, e.g., the ideal gas constant p_{23} , being unknown.

Table 2

Coefficient values for the economic profit indicator P (the former Dutch currency Hfl, the Guilder, is used for adherence to the literature [4,22,27–29]).

Symbol	Value	Unit
$c_{\text{price},1}$	1.8	Hflm ⁻²
$c_{\text{price},2}$	1.6	Hflkg ⁻¹
c_q	$6.35 \cdot 10^{-9}$	HflJ ⁻¹
c_{CO_2}	$42 \cdot 10^{-2}$	Hflkg ⁻¹
Δt	900	s

This performance indicator captures the magnitude of violations on the output constraints over a growth cycle. Finally, the economic profit indicator³ P is defined as [29,22]

$$P = c_{\text{price},1} + c_{\text{price},2} y_1(N_s) - \sum_{k=0}^{N_s} (c_{\text{CO}_2} u_1(k) + c_q u_3(k)) \Delta t, \quad (16)$$

where the relation between auction price and harvest weight of lettuce is modeled linearly with coefficients $c_{\text{price},1}$ and $c_{\text{price},2}$, and the financial cost of the climate conditioning equipment is linearly related to the amount of energy and carbon dioxide put into the system, weighted by prices c_q and c_{CO_2} , respectively. Note that in P no cost is associated to natural ventilation used for cooling and dehumidification, i.e., u_2 [8]. The values of the coefficients of this economic model are given in Table 2, and more details are available in, e.g., [29, Section 2.1]. This performance indicator represents the monetary value of a growth cycle, and captures the efficiency of the control with respect to the use of costly actuators.

4. Methodology

To address the issues caused by inaccurate knowledge of the true prediction model parameters p for model-based MPC controllers, we propose a novel parametrized MPC scheme for the greenhouse climate control problem. Then, leveraging closed loop data, we show how the parameter values can be learned using RL techniques [9], compensating for the performance loss introduced by an inaccurate prediction model.

4.1. Greenhouse climate control as an RL problem

In order to apply an RL methodology to learn the MPC parametrization, the greenhouse climate control must be modeled as an RL task, i.e., as an MDP defined as in Section 2.2. Trivially, the greenhouse input variable u corresponds directly to the actions a in the RL context. Define the concatenation of the current state, and the current and previous outputs, as the RL state:

$$s(k) = (x^\top(k), y^\top(k), y^\top(k-1))^\top. \quad (17)$$

Furthermore, the state transitions are determined by the true system model (1), where the exact probabilities are conditioned on the weather disturbance d . Note that, to enforce the control rate constraint (14) in the MDP, the input variable is clipped prior to being applied to the system.

The stage cost L requires particular attention, as it implicitly defines the optimization problem (4) that the RL agent is tasked with solving. Crafting the stage cost L such that the resulting RL policy performs well with respect to the performance indicators defined in Section 3 is a design challenge. Consider the stage cost

³ Note that P represents a profit indicator per square meter, as the lettuce model in this work is normalized by the surface area (see Section 2.1 and [28, 29]).

$$L(s(k), a(k)) = L_{y_1}(s(k)) + L_u(a(k)) + L_\Psi(s(k)). \quad (18)$$

The function

$$L_{y_1}(s(k)) = -c_{\delta y_1}(y_1(k) - y_1(k-1)), \quad (19)$$

with $c_{\delta y_1} > 0$, rewards the step-wise increase in dry lettuce weight. The function

$$L_u(a(k)) = c_u^\top u(k), \quad (20)$$

with $c_u \in \mathbb{R}^3 > 0$, penalizes the control inputs, while the function

$$L_\Psi(s(k)) = \omega^\top \max \left(0, \frac{y(k) - y_{\max}(k)}{y_{\max}(k) - y_{\min}(k)} \right) + \omega^\top \max \left(0, \frac{y_{\min}(k) - y(k)}{y_{\max}(k) - y_{\min}(k)} \right) \quad (21)$$

penalizes constraint violations, with $\omega \in \mathbb{R}^4 > 0$, and with the max operator and vector division performed element-wise. The constants $c_{\delta y_1}$, c_u , and ω are then hyper-parameters that are tuned such that the RL policy performs well on the performance indicators in Section 3. Lastly, the RL policy π_θ , and value functions V_θ and Q_θ , are addressed via the parametrized MPC scheme, which is introduced next.

4.2. Parametrized MPC scheme

In this section we introduce the parametrized MPC scheme that, following the theory outlined in Section 2.2, serves as policy provider and value function approximator for the RL task outlined in Section 4.1.

Consider the following parametrized MPC scheme, a concrete realization of (6), representing the value function, with prediction horizon $N \in \mathbb{Z}^+$ and discount factor $\gamma \in (0, 1]$:

$$\begin{aligned} V_\theta(s) = & \min_{\mathbf{y}, \mathbf{x}, \mathbf{u}, \boldsymbol{\sigma}} \theta_0 - \theta_{\delta y_1} \sum_{k=1}^N \gamma^k (y_1(k) - y_1(k-1)) \\ & + \theta_u^\top \sum_{k=0}^{N-1} \gamma^k u(k) + \theta_\omega^\top \sum_{k=0}^N \gamma^k \frac{\sigma(k)}{y_{\text{range}}} \\ & + \theta_{y_1, f} \gamma^N V_{\theta, f}(y_1(N)) \end{aligned} \quad (22a)$$

$$\text{s.t.} \quad x(0) = x, \quad (22b)$$

$$\text{for } k = 0, \dots, N-1$$

$$x(k+1) = f(x(k), u(k), d(k), \theta_p), \quad (22c)$$

$$u_{\min} \leq u(k) \leq u_{\max}, \quad (22d)$$

$$-\delta u \leq u(k) - u(k-1) \leq \delta u, \quad (22e)$$

$$\text{for } k = 0, \dots, N$$

$$y(k) = g(x(k), \theta_p), \quad (22f)$$

$$y_{\min}(k) - \sigma(k) \leq y(k) \leq y_{\max}(k) + \sigma(k), \quad (22g)$$

$$\sigma(k) \geq 0, \quad (22h)$$

where⁴ $y_{\text{range}} = (\infty, 1.6, 5, 70)^\top$. The cost (22a) rewards step-wise lettuce weight increase, and penalizes constraint violations and control inputs. The first and last terms, θ_0 and $V_{\theta, f}$, are additional initial offset and terminal costs respectively, enriching the parametrization to help the MPC scheme capture the true RL value functions. In particular, θ_0 is required in order to learn the (possibly non-zero) offset in the value function due to its economic nature (see [9] for more theoretical details), while $V_{\theta, f}$ encodes the cost-to-go. The parameters $\theta_{\delta y_1}$, θ_u , θ_ω , and $\theta_{y_1, f}$ weight the contributions of each cost term. Since in general $\theta_p \neq p$,

the constraints for the dynamics and output equations, (22c) and (22f), are parametrized by θ_p in place of the true model parameters p . Hence, the predicted state and output trajectories from solving (22) do not necessarily respect the dynamics and output functions of the true system. The parameter θ_p then provides a degree of freedom to optimize the predicted trajectories for closed-loop performance, using closed-loop data.

Remark 1. Note that the MPC parameters $\theta_{\delta y_1}$, θ_u , θ_ω , and θ_p are distinct from the parameters in the RL stage cost $c_{\delta y_1}$, c_u , ω , and the true model parameters p . While they represent equivalent notions, the MPC parameters will be adjusted to optimize closed-loop performance and will not, in general, converge to the corresponding values in the RL stage-cost and the true model. Indeed, in [9] it is stressed that the prediction model yielding the best closed-loop performance for an MPC controller is not necessarily the one with the smallest prediction error. Hence, even if instantiated with accurate model parameters $\theta_p = p$, it is likely the RL agent would adjust θ_p regardless, optimizing for closed-loop performance.

The terminal cost $V_{\theta, f}$ is added to capture the future reward from lettuce weight increase for the remainder of the growth cycle, occurring after the prediction horizon of the MPC controller. The step-wise reward for lettuce growth for the remainder of the cycle can be simplified as

$$\sum_{k=N+1}^{N_s} \gamma^k L_{y_1}(s(k)) \quad (23)$$

$$\begin{aligned} &= -c_{\delta y_1} \sum_{k=N+1}^{N_s} \gamma^k (y_1(k) - y_1(k-1)) \\ &= -c_{\delta y_1} \left(\gamma^{N_s} y_1(N_s) - \gamma^{N+1} y_1(N) \right. \\ &\quad \left. + \sum_{k=N+1}^{N_s-1} (\gamma^k - \gamma^{k+1}) y_1(k) \right), \end{aligned} \quad (24)$$

$$\approx -c_{\delta y_1} (y_1(N_s) - y_1(N)), \quad (25)$$

where, for the sake of simplicity, it is assumed that $\gamma \approx 1$, which is common when a non-myopic policy is desired [26]. Of course, (25) cannot be used as terminal cost directly, as the final yield $y_1(N_s)$ is not known and varies depending on the weather disturbance and the control inputs. As the value functions attempt to capture the expected cost under the current policy (see (5)), we introduce the learnable parameter θ_{y_N} in order to learn the expected value $\mathbb{E}_{\eta_{\pi_\theta}} [y_1(N_s)]$, where, again, η_{π_θ} is the state distribution induced by the policy π_θ and the weather disturbance d . To capture (25), the terminal cost is then defined as

$$V_{\theta, f}(y_1(N)) = -\theta_{\delta y_1} (\theta_{y_N} - y_1(N)). \quad (26)$$

The full parametrization is then

$$\theta = (\theta_0, \theta_{\delta y_1}, \theta_u^\top, \theta_\omega^\top, \theta_{y_1, f}, \theta_{y_N}, \theta_p^\top)^\top. \quad (27)$$

The allowable range for the parameters θ can be bounded during learning, incorporating prior knowledge on viable ranges, or enforcing realistic values on parameters that have physical significance. Table 3 gives a summary of the parametrization, while the bounds and initialization of the parameters in our experiments is given in Section 5.

As outlined in Section 2.2, the MPC scheme (22) for the value function approximation $V_\theta(s)$ satisfies the Bellman equalities [9], such that the action-value function and policy are delivered by the same scheme as

$$\begin{aligned} Q_\theta(s, a) = & \min_{\mathbf{y}, \mathbf{x}, \mathbf{u}, \boldsymbol{\sigma}} \quad (22a) \\ \text{s.t.} \quad & (22b) - (22h), \end{aligned} \quad (28)$$

$$u(0) = a,$$

⁴ The ∞ in y_{range} causes the first component of the violation penalty to be always zero, as the first output is unbounded.

Table 3
Summary of MPC parametrization in (22).

Symbol	Scope	Space
θ_0	cost - offset	$\mathbb{R}$
$\theta_{\delta y_1}$	cost - weight reward	$\mathbb{R}$
θ_u	cost - control penalty	$\mathbb{R}^3$
θ_ω	cost - violations penalty	$\mathbb{R}^4$
$\theta_{y_1, f}$	cost - terminal cost weight	$\mathbb{R}$
θ_{y_N}	cost - terminal weight estimate	$\mathbb{R}$
θ_p	model parameters	$\mathbb{R}^{28}$

$$\pi_\theta(s) = \arg \min_u Q_\theta(s, u). \quad (29)$$

Instead of (29), the equivalent form

$$\begin{aligned} \pi_\theta(s) = \arg \min_{u(0)} \quad & (22a) \\ \text{s.t.} \quad & (22b) - (22h), \end{aligned} \quad (30)$$

is used in practice such that both $V_\theta(s)$ and $\pi_\theta(s)$ are found by solving the one optimization problem (22).

4.3. Second-order LSTD Q-learning

We apply a second-order least-squares temporal difference (LSTD) Q-learning algorithm [16] to learn the parametrization θ of (22), such that the policy π_θ optimizes the greenhouse climate control RL task, as defined in Section 4.1. Q-learning attempts to learn a parametrization θ such that the action-value function Q_θ fits the observed data. The policy is then inferred from the action-value function as $\pi_\theta(s) = \arg \min_a Q_\theta(s, a)$. In this work the Q-learning algorithm is applied on-line, i.e., training data is generated via interaction with the real system, as the policy is trained. Reformulating the fitting of Q_θ as a least-squares problem, in combination with a second-order Newton's method and an experience replay buffer of the past observed transitions [18], provides faster convergence and better sample efficiency with respect to traditional first-order methods. This approach has been effectively applied in the context of MPC [7,1].

As in [1], we impose lower and upper bounds on the parameters with the following constrained optimization problem

$$\begin{aligned} \Delta\theta^* = \arg \min_{\Delta\theta} \quad & \frac{1}{2} \Delta\theta^\top \bar{H} \Delta\theta + \alpha \bar{g}^\top \Delta\theta \\ \text{s.t.} \quad & \theta_{lb} \leq \theta + \Delta\theta \leq \theta_{ub}, \\ & \Delta\theta_{lb} \leq \Delta\theta \leq \Delta\theta_{ub}, \end{aligned} \quad (31)$$

with $\alpha > 0$ the learning rate, and $\bar{g}$ and $\bar{H}$ the gradient and Hessian of the Q-learning fitting problem averaged over m samples drawn from the replay buffer, i.e.,

$$\bar{g} = - \sum_{i=1}^m \delta_i \nabla_\theta Q_\theta(s_i, a_i), \quad (32)$$

$$\bar{H} = \sum_{i=1}^m \nabla_\theta Q_\theta(s_i, a_i) \nabla_\theta Q_\theta^\top(s_i, a_i) - \delta_i \nabla_\theta^2 Q_\theta(s_i, a_i). \quad (33)$$

Note that the sensitivities $\nabla_\theta Q_\theta$ and $\nabla_\theta^2 Q_\theta$ are available automatically upon solving (28); see [1,7] for details. This formulation additionally allows to limit the rate of change of each parameter with $\Delta\theta_{lb}$ and $\Delta\theta_{ub}$. Finally, the parametrization θ is then updated as $\theta \leftarrow \theta + \Delta\theta^*$.

Note that during training of the policy two optimization problems are solved at each time step: (22) for V_θ and π_θ , and (28) for Q_θ , in order to be able to compute and store in buffer the TD error and sensitivities necessary for an update. Furthermore, the quadratic program (31) must be solved only once per parameter update. In contrast, when the policy is not being trained, only (22) must be solved for π_θ .

Table 4
Initial values and bounds on parametrization in (22).

Symbol	Initial value	Bounds
θ_0	0	$(-\infty, \infty)$
$\theta_{\delta y_1}$	100	$(0, \infty)$
θ_u	$(10, 1, 1)^\top$	$(0, \infty)$
θ_ω	$(10^5, 10^5, 10^5, 10^5)^\top$	$(0, \infty)$
$\theta_{y_1, f}$	1	$(0, \infty)$
θ_{y_N}	135	$(0, \infty)$
θ_p	$\sim \mathcal{R}$	$(0.5 p , 1.5 p)$

Table 5
Coefficients in the RL stage cost.

Symbol	Value
$c_{\delta y_1}$	100
c_u	$(10, 1, 1)^\top$
ω	10^5

Remark 2. In general, problem (4) is nonlinear and non-convex. This implies that the RL algorithm is likely to converge to local suboptimal solutions. To alleviate this issue, exploratory behavior can be injected into the learning policy in an effort to escape such local minima and to converge to a better (possibly global) solution. One method is to perturb the gradient of the objective of Q_θ in an, e.g., ϵ -greedy fashion [1]. In the current work, exploration is instead induced during the learning process by the weather forecast disturbances which, as explained in more detail in Section 5.1, are stochastically generated at the beginning of each training episode. As later shown in Section 5.3, this is enough to provide satisfactory convergence of the TD error.

5. Numerical experiments

In this section the methodology proposed in Section 4 is demonstrated in simulation, with the resulting performance compared against existing state-of-art approaches. In the following, all MPC simulations are run on a Linux machine using one AMD EPYC 7252 core, 1.38 GHz clock speed, and 251Gb of RAM. All simulations for training DNN-based RL methods, i.e., DDPG, are run on the same Linux machine using four NVIDIA RTX 2090 GPUs. Python source code and simulation results can be found at <https://github.com/SamuelMallick/mpcrl-greenhouse>. All optimization problems are solved using the CasADi framework [2] and the IPOPT solver [31].

Assume that the values of all model parameters p are unknown. What is known is an uncertainty range $[p - 0.2 \cdot |p|, p + 0.2 \cdot |p|]$ that contains the true values, where $|\cdot|$ is the element-wise absolute value, and define a uniform distribution spanning the uncertainty range as

$$\mathcal{R} = \mathcal{U}(p - 0.2 \cdot |p|, p + 0.2 \cdot |p|). \quad (34)$$

The initial values for the learnable model parameters are then a random sample from $\mathcal{R}$, and they are bounded to an enlargement of the uncertainty range. To initialize the cost parameters, the values from the RL stage cost are used. Further, we bound all cost terms to be non-negative, such that the notion of rewarded and penalized behavior does not invert. The initial values and bounds for the full MPC parametrization are given in Table 4. The RL stage cost coefficients used in our experiments are given in Table 5.

5.1. Weather disturbances

The weather disturbance profile d used in the experiments is real weather data presented in [13], collected from ‘the Venlow Energy greenhouse’, located in Bleiswijk, The Netherlands. The data covers a full growth cycle of 40 days and, originally sampled at 5 minute inter-

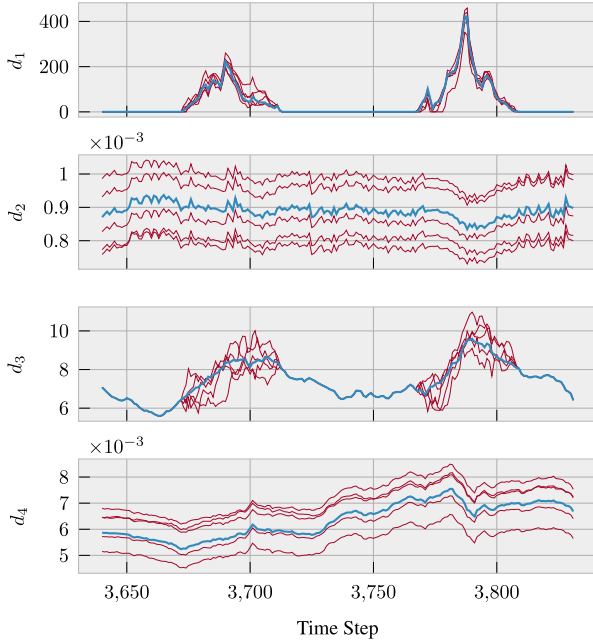


Fig. 2. Last two days of weather profiles. The blue profile is the original, and the red lines are five example perturbed profiles.

vals, has been resampled using an FIR anti-aliasing low pass filter at the sampling time used in this work, i.e., 15 minutes [4].

As RL approaches train on repeated growth cycle episodes, to avoid over-fitting of the learned policy to a specific instance of weather data and to foster generalization over a wider range of weather disturbances, a stochastic process is added to the real weather data in order to generate a new perturbed profile for each episodic growth cycle. In particular, Brownian noise, a time-correlated stochastic process, is added to the original weather profile. The Brownian noise is generated as the cumulative sum of white noise as

$$B(k) = \sum_{\tau=0}^k W(\tau), \quad W(\tau) \sim \mathcal{U}(-\rho, \rho), \quad (35)$$

where $\rho = (0.01, 0.005, 0.01, 0.005)^\top$, and $B(k)$ is the noise value at time step k . For the outdoor CO_2 level (d_2) and the outdoor humidity (d_4), this noise is added directly to the weather data. For the radiation (d_1) and the outdoor temperature (d_3), special care is required to ensure the perturbed weather profiles still follow a reasonable day/night cycle. To achieve this, Brownian excursions [30], stochastic processes that have constrained initial and final values, are used such that the Brownian noise only takes effect during the day cycle, and arrives at the relative night-time values at the start of the night cycle. Fig. 2 demonstrates the original, and five perturbed, weather disturbance profiles for the last two days of the growth cycle.

5.2. Comparison approaches

We compare our approach, referred to as MPC-RL, against the robust-sample based MPC controller from [4], and a model-free RL controller trained with DDPG [22,32]. Furthermore, we include two nominal MPC controllers for comparison. One is ideal, with perfect (but unrealistic) model knowledge, while the other is a nominal MPC controller using an incorrect prediction model. All MPC controllers in the following use a horizon of $N = 24$. This value is selected to balance performance and computation time [4,22]. In particular, the comparison approaches are:

- **Ideal MPC controller (I-MPC):** The ideal MPC controller is a standard MPC controller with knowledge of the true dynamics via the real values of p .

Table 6
Hyper-parameters for the DDPG RL Controller.

Parameter	Value
learning rate	10^{-5}
gradient threshold	1
L_2 regularization factor	10^{-5}
experience buffer size	10^4
experience mini-batch size	64

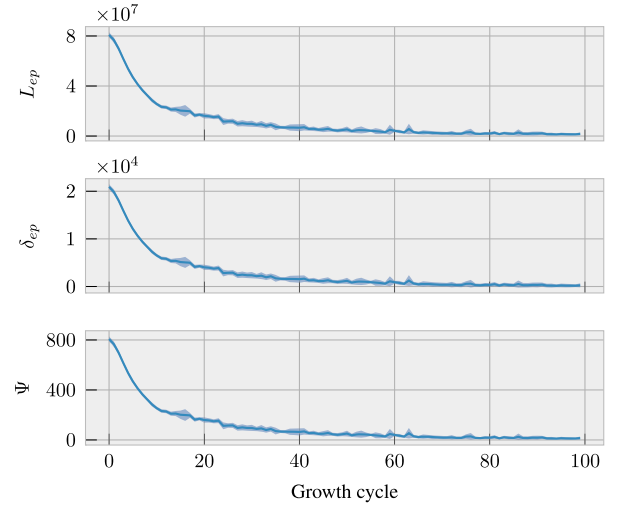


Fig. 3. Stage cost, TD error, and constraint violations over 100 growth cycles of training.

- **Nominal MPC controller (N-MPC):** The nominal MPC controller is a standard MPC controller using an incorrect prediction model with parameters $\hat{p} \sim \mathcal{R}$.
- **Robust MPC controller (R-MPC- n) [4]:** The robust sample-based MPC controller is essentially an implementation of a stochastic MPC controller based on the scenario approach [23]. The parametric model uncertainty is addressed by sampling the probability distribution of model parameters, in this case the uniform distribution $\mathcal{R}$, and optimizing one state trajectory for each sample in the MPC optimization. In [4], 20 samples were used. In our experiments we test a range of numbers of samples n , indicating the number of samples in the shorthand name, e.g., 5 samples is denoted R-MPC-5.
- **DDPG RL controller (DDPG) [22,32]:** DDPG is a model-free off-policy RL algorithm that leverages DNNs as function approximators. Conversely to the proposed Q-learning algorithm, this method belongs to the policy gradient family, which optimizes the parametrization via estimates of the gradient of the policy w.r.t. θ [17]. The learning hyper-parameters are taken from [22] and are reported in Table 6.

Note that both learning-based and non-learning-based controllers described above are deployed in simulations on the same greenhouse environment. Obviously, this environment employs the correct model to simulate the evolution of the real dynamics and to generate training data.

5.3. Results

We run 100 growth cycles to learn the parametrization of the MPC scheme (22), with the initial parametrization in Table 4. We simulate online learning, where the data generated while interacting with the system is used to learn the policy. The learning is concluded after 100 episodes as, for this case study, this is sufficient to observe satisfactory convergence of the TD error and the learned MPC parametrization,

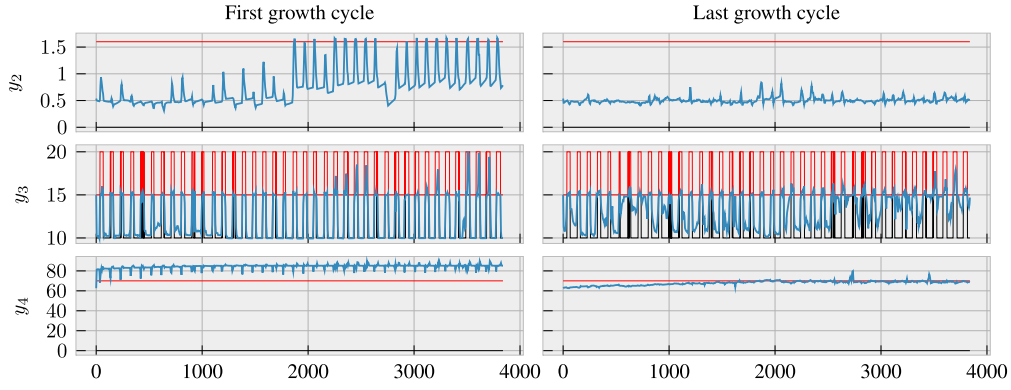


Fig. 4. Outputs over first and last growth cycle of training with upper (red lines) and lower (black lines) bounds.

alongside improvements to both the economic profit and constraint satisfaction. The hyper-parameters of the learning process are tuned as follows. The discount factor is selected as $\gamma = 0.99$, while the learning rate is $\alpha = 0.1$. The parameters are updated once at the end of each growth cycle, with the maximum update being constrained to 5% of their current values, i.e., $\Delta\theta_{lb} = -0.05|\theta|$ and $\Delta\theta_{ub} = 0.05|\theta|$. A replay buffer stores the observations from the last three cycles, and every update considers two cycles worth of observations, with all observations from the most recent cycle guaranteed to be used. Three different parametrizations are learned for both RL-based approaches, where, for each, the random seed for generating the weather profiles is different. The initial state for each cycle is $x(0) = (0.0035 \ 0.001 \ 15 \ 0.008)^T$.

Fig. 3 shows the episode-wise stage cost $L_{ep} = \sum_{k=0}^{N_s} L(s(k), u(k))$, the TD error $\delta_{ep} = 1/N_s \sum_{k=0}^{N_s} \delta(k)$, and the constraint violations Ψ for each growth cycle during training. It can be seen that the incurred stage cost and TD error are significantly reduced during training. This is further represented in the constraint violations, which, starting from a large value, approach zero. Fig. 4 shows the constrained outputs during the first and last growth cycles of training. It can be seen that in the first growth cycle, the indoor CO₂ upper bound (y_2) is violated several times towards the end of the cycle, while the indoor humidity upper bound (y_4) is violated for practically the entire cycle. After 100 cycles of learning, it can be seen that the indoor CO₂ level never violates the bounds, and the indoor humidity level exceeds the upper bound only a few times, quickly dropping back to allowable levels. Fig. 5 shows the parameter evolution over the training for a subset of the learnable parameters; the four parameters whose values changed the most. Finally, in Appendix A the control inputs during the first and last growth cycles of training are included for completeness.

In an evaluation phase, i.e., with the policy fixed, we compare our final trained RL-based MPC controller against the approaches outlined in Section 5.2. Each approach is evaluated over 100 growth cycles, with Fig. 6 demonstrating the results with respect to the performance indicators given in Section 3. The episode-wise stage cost is included for interest, especially for comparison between the two RL-based approaches, that during training learn policies to minimize this cost, as in (4). For completeness, the state, output, and input trajectories for the final 10 days of an evaluation growth cycle are included in Appendix A.

The constraint violations Ψ show that our approach has reduced the constraint violations the most, approaching the level of the ideal MPC controller. The robust MPC controller does improve the constraint violations over the nominal MPC; however, they remain at an unsatisfactory level. Finally, the DDPG-based controller improves the violations over both the nominal and robust MPC controllers, but does not reach that of our approach.

The crop yield $y_1(N_s)$ shows that the controllers with very high constraint violations, i.e., the nominal MPC controller and robust MPC controller with 5 samples, generate a very high crop yield. This is in line with observations in the literature [34], and is due to the unrealis-

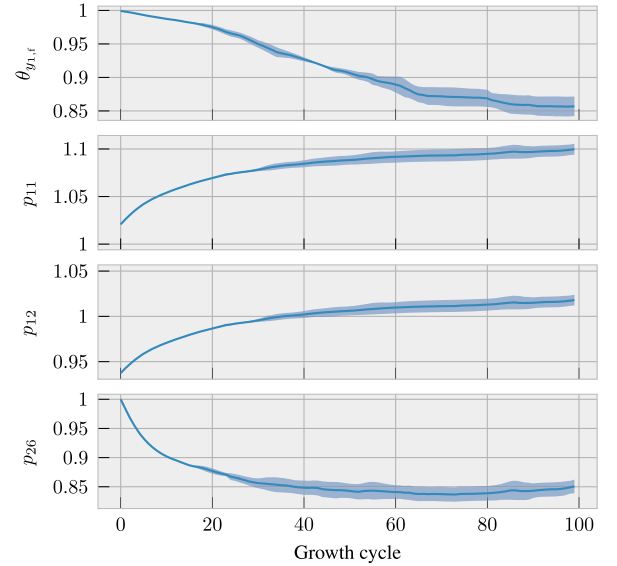


Fig. 5. Parameter evolution, for the 4 most changed parameters, over 100 growth cycles of training. Note that model parameters θ_p have been normalized with respect to their true values.

tic growth that occurs in the model when variables such as CO₂ levels and humidity are dangerously high. Notably, for higher numbers of samples, the robust MPC controllers significantly reduce the crop yield due to conservatism. Our approach has reduced the drop in crop yield, even while vastly improving the constraint violations. The DDPG controller gives superior crop yield even with respect to the ideal MPC controller. This is again in line with observations in the literature [22], and is due to the DDPG approach's tendency to optimize the crop yield over efficiency and constraint violations.

The economic profit indicator P shows similar trends to the crop yield for all MPC-based controllers. Again, we highlight that the high economic profit of the nominal MPC controller, and the robust MPC controller with 5 samples, is due to the unrealistic growth occurring in the model when the output constraints are significantly violated. A clear difference from the crop yield trend is for the DDPG controller, which now under-performs in comparison to both the ideal MPC controller and our approach. This demonstrates how the DDPG controller maximizes the yield in an inefficient way with respect to the control inputs, resulting in an overall reduced profit.

Finally, with respect to the stage cost L_{ep} , our approach outperforms the DDPG controller. This is notable as, during training, both RL-based approaches attempt to solve an optimization problem that minimizes L_{ep} , i.e., the performance (3).

Table 7 shows the average computation time needed per time step for each of the approaches. Naturally, the DDPG approach is the fastest

Table 7
Average computation time required by each approach.

Approach	I	N	R5	R10	R20	MPC-RL training	MPC-RL evaluation	DDPG
Time (s)	0.0378	0.0390	0.229	0.570	1.234	0.0793	0.0430	0.000155

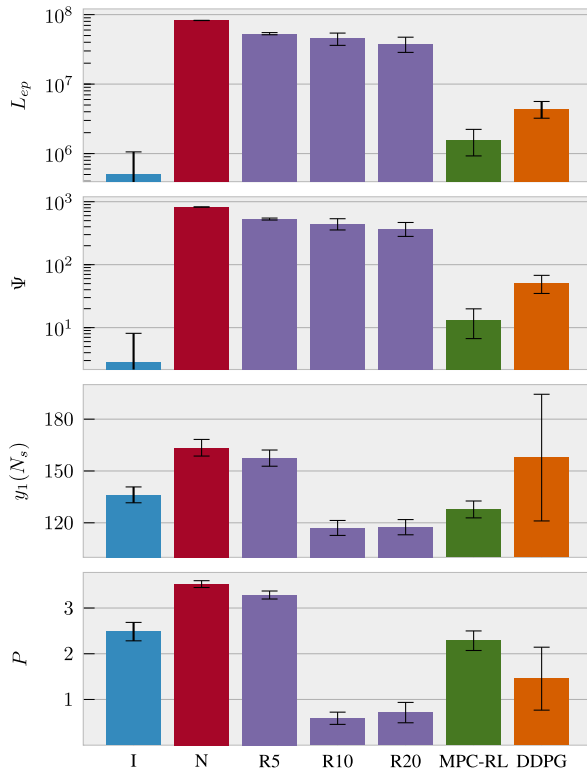


Fig. 6. Comparison of all approaches over 100 growth cycles. The learning-based strategies are tested after convergence. Bars show the mean value, with error bars showing a standard deviation.

as no optimization problem is solved when computing its actions. Comparing the MPC-based approaches, our approach requires similar computation times to those of the I-MPC and N-MPC approaches during evaluation, and is only slightly slower during training. In contrast, the R-MPC approaches introduce additional optimization variables with each additional sample, and the required computation time increases as the samples increase.

6. Conclusions

In this work, we propose an RL-based MPC controller for greenhouse climate control. At the core of the approach is a parametrized MPC scheme that can serve as policy provider and value-function approximator for an RL task. The greenhouse climate control problem has been formulated as an RL task, such that the MPC scheme can learn a parametrization online using closed-loop data. Second-order Q-learning has been proposed as the RL algorithm for learning the parametrization. In simulations, the proposed approach has been shown to learn an MPC scheme that significantly reduces the violations of output constraints in closed-loop operation. The final learned controller has then been compared against state-of-the-art MPC- and RL-based controllers from the literature, showing the best performance in terms of constraint violations and efficient crop growth.

Future work directions include the application of this methodology to alternative greenhouse models, and experimental validation in real-world tests. Additionally, alternative learning algorithms, such as policy

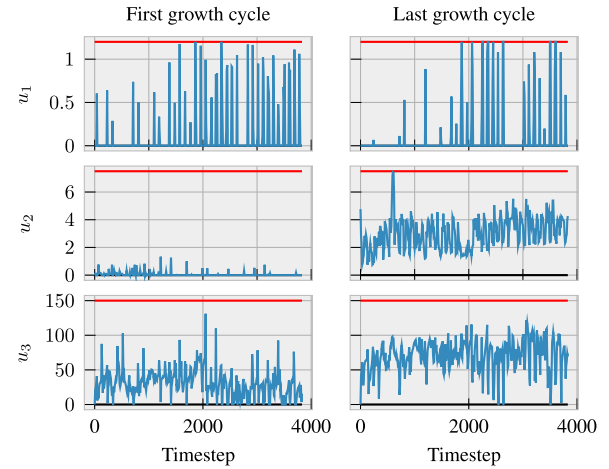


Fig. 7. Control inputs for the first and last growth cycle of training with upper (red lines) and lower (black lines) bounds.

gradient approaches, could be explored to learn the parametrization of the MPC scheme.

CRedit authorship contribution statement

Samuel Mallick: Writing – original draft, Software, Methodology, Investigation, Conceptualization. **Filippo Airdi:** Writing – original draft, Software, Methodology, Investigation, Conceptualization. **Azita Dabiri:** Writing – review & editing, Supervision, Methodology, Investigation, Conceptualization. **Congcong Sun:** Writing – review & editing, Supervision, Conceptualization. **Bart De Schutter:** Writing – review & editing, Supervision, Conceptualization.

Funding

This paper is part of a project that has received funding from the European Research Council (ERC) under the European Union's Horizon 2020 research and innovation programme (Grant agreement No. 101018826 - CLariNet).

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Appendix A. Extra input, output, and state trajectories

Fig. 7 shows the control inputs of our approach during the first and last episodes of training. The trajectories are clearly different, as the approach has learned a policy that optimizes for closed-loop performance.

Figs. 8, 9, and 10 show the output, state, and input trajectories, respectively, for the final 10 days of a growth cycle during evaluation of the compared controllers. Notably, the additional constraint violations in the outputs, introduced by the comparison controllers, are evident. Further, the control inputs demonstrate the inefficiency of the DDPG controller's actuation.

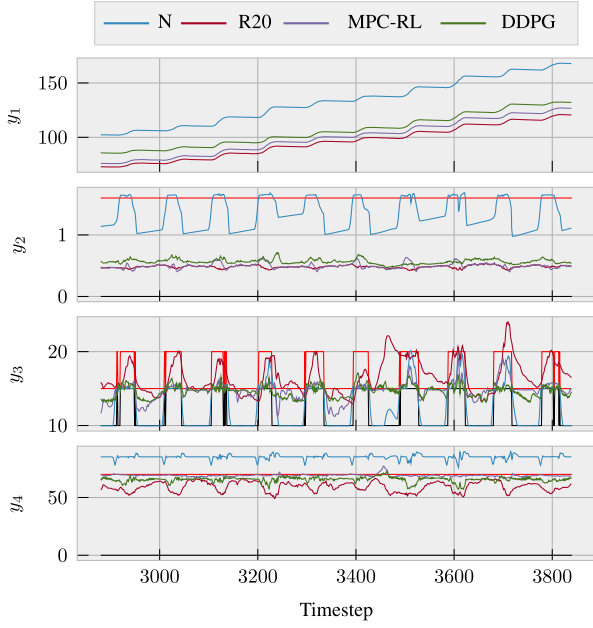


Fig. 8. Output trajectories for the last 10 days of a growth cycle during evaluation, with upper (red lines) and lower (black lines) bounds.

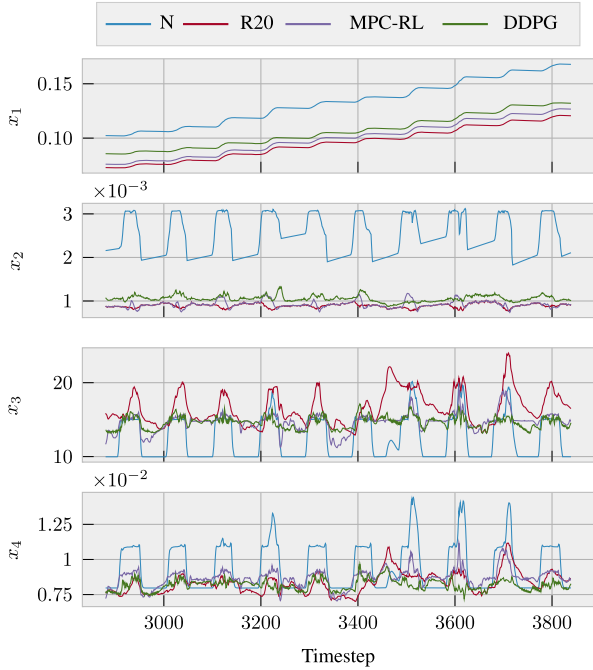


Fig. 9. State trajectories for the last 10 days of a growth cycle during evaluation.

Appendix B. Greenhouse model

The continuous-time lettuce growing greenhouse model is described by the following set of equations [28]

$$\begin{aligned}\dot{x}_1(t) &= p_1 \phi_{\text{phot},c}(t) - p_2 x_1(t) 2^{x_3(t)/10-5/2}, \\ \dot{x}_2(t) &= \frac{1}{p_9} \left(-\phi_{\text{phot},c}(t) + p_{10} x_1(t) 2^{x_3(t)/10-5/2} \right. \\ &\quad \left. + 10^{-6} u_1(t) - \phi_{\text{vent},c}(t) \right),\end{aligned}$$

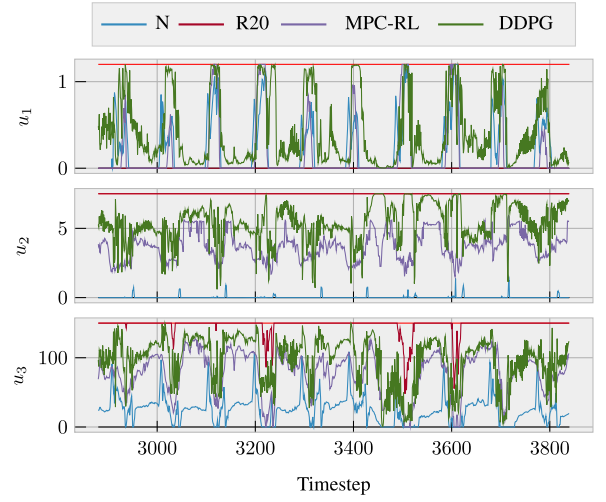


Fig. 10. Input trajectories for the last 10 days of a growth cycle during evaluation, with upper (red lines) and lower (black lines) bounds.

$$\begin{aligned}\dot{x}_3(t) &= \frac{1}{p_{16}} \left(u_3(t) - (10^{-3} p_{17} u_2(t) + p_{18}), \right. \\ &\quad \left. \cdot (x_3(t) - d_3(t)) + p_{19} d_1(t) \right),\end{aligned}$$

$$\dot{x}_4(t) = \frac{1}{p_{20}} (\phi_{\text{transp},h}(t) - \phi_{\text{vent},h}(t)),$$

$$y_1(t) = 10^3 x_1(t),$$

$$y_2(t) = 10^3 x_2(t) \frac{p_{12}(x_3(t) + p_{13})}{p_{14} p_{15}},$$

$$y_3(t) = x_3(t),$$

$$y_4(t) = \frac{10^2}{11} x_4(t) \frac{p_{12}(x_3(t) + p_{13})}{\exp\left(\frac{p_{27} x_3(t)}{x_3(t) + p_{28}}\right)},$$

where $t \geq 0$ is continuous time. Further, define the following functions:

$$\begin{aligned}\phi_{\text{phot},c}(t) &= \frac{1}{\varphi(t)} \left(1 - \exp(-p_3 x_1(t)) \right) \\ &\quad \cdot \left(p_4 d_1(t) (-p_5 x_3(t)^2 + p_6 x_3(t) - p_7) \right. \\ &\quad \left. \cdot (x_2(t) - p_8) \right),\end{aligned}$$

$$\varphi(t) = p_4 d_1(t) + (-p_5 x_3(t)^2 + p_6 x_3(t) - p_7)$$

$$\cdot (x_2(t) - p_8),$$

$$\phi_{\text{vent},c}(t) = (u_2(t) 10^{-3} + p_{11}) (x_2(t) - d_2(t)),$$

$$\phi_{\text{transp},h}(t) = p_{21} \left(1 - \exp(-p_3 x_1(t)) \right)$$

$$\cdot \left(\frac{p_{22}}{p_{23}(x_3(t) + p_{24})} \exp\left(\frac{p_{25} x_3(t)}{x_3(t) + p_{26}}\right) - x_4(t) \right),$$

$$\phi_{\text{vent},h}(t) = (u_2(t) 10^{-3} + p_{11}) (x_4(t) - d_4(t)),$$

where $\phi_{\text{phot},c}(t)$, $\phi_{\text{vent},c}(t)$, $\phi_{\text{transp},h}(t)$ and $\phi_{\text{vent},h}(t)$ are the gross canopy photosynthesis rate, mass exchange of CO_2 through the vents, canopy transpiration and mass exchange of H_2O through the vents, respectively.

To generate the discrete-time prediction mode (1), discretization is performed using the explicit fourth order Runge-Kutta method with a time step of 15 minutes, as in [4]. Finally, the values for the model parameters p are given in Table 8.

Data availability

Data will be made available on request.

Table 8
Model parameters.

Symbol	Value	Unit	Explanation from [28,29]
p_1	0.544	-	Yield factor; aggregation of c_a (CO ₂ -glucose stoichiometric conversion factor) and c_β (yield of carbohydrates conversion to structural material)
p_2	$2.65 \cdot 10^{-7}$	s ⁻¹	Aggregation of $c_{resp,s}$ and $c_{resp,r}$ (maintenance respiration rates for shoot and root at 25 °C) weighted by c_β and c_r (ratio of root dry weight to total crop dry weight)
p_3	53	m ² kg ⁻¹	Effective canopy surface; aggregation of $c_{lar,d}$ (shoot leaf area ratio) with c_k (canopy extinction coefficient) and c_r
p_4	$3.55 \cdot 10^{-9}$	kg J ⁻¹	Aggregation of c_{par} (photosynthetically activate radiation to total solar radiation ratio) and $c_{rad,rf}$ (roof solar radiation transmission coefficient)
p_5	$5.11 \cdot 10^{-6}$	m s ⁻¹ °C ⁻²	Second-order term in polynomial approximation of temperature effect on CO ₂ leaf diffusion
p_6	$2.3 \cdot 10^{-4}$	m s ⁻¹ °C ⁻¹	First-order term in polynomial approximation of temperature effect on CO ₂ leaf diffusion
p_7	$6.29 \cdot 10^{-4}$	m s ⁻¹	Zeroth-order term in polynomial approximation of temperature effect on CO ₂ leaf diffusion
p_8	$5.2 \cdot 10^{-5}$	kg m ⁻³	CO ₂ compensation point at 25 °C
p_9	4.1	m	Volumetric capacity of greenhouse air for CO ₂
p_{10}	$4.87 \cdot 10^{-7}$	s ⁻¹	Aggregation of $c_{resp,s}$ and $c_{resp,r}$ weighted by c_a and c_r
p_{11}	$7.5 \cdot 10^{-6}$	m s ⁻¹	Leakage air exchange through greenhouse cover
p_{12}	8.314	JK ⁻¹ mol ⁻¹	Gas constant
p_{13}	273.15	K	Absolute temperature
p_{14}	101325	Pa	Sea level standard atmospheric pressure
p_{15}	0.044	kg mol ⁻¹	CO ₂ molar mass
p_{16}	$3 \cdot 10^4$	J m ⁻² °C ⁻¹	Heat capacity of greenhouse air
p_{17}	1290	J m ⁻³ °C ⁻¹	Heat capacity per volume unit of greenhouse air
p_{18}	6.1	W m ⁻² °C ⁻¹	Heat transfer coefficient through greenhouse cover
p_{19}	0.2	-	Sun heat load coefficient accounting for roof transmission, solar radiation interception by structural components, and conversion from absorbed solar energy to sensible heat load by canopy
p_{20}	4.1	m	Volumetric capacity of greenhouse air for humidity
p_{21}	0.0036	m s ⁻¹	Canopy transpiration mass transfer coefficient
p_{22}	9348	J kg m ⁻³ kmol ⁻¹	Aggregation of c_{H_2O} (water molar mass), $c_{v,0}$ (calibration parameter) and $c_{v,1}$ (saturation water vapor pressure parameter for canopy transpiration)
p_{23}	8314	JK ⁻¹ kmol ⁻¹	Gas constant
p_{24}	273.15	K	Absolute temperature
p_{25}	17.4	-	Saturation water vapor pressure parameter for canopy transpiration
p_{26}	239	°C	Saturation water vapor pressure parameter for canopy transpiration
p_{27}	17.269	-	Saturation water vapor pressure parameter for humidity
p_{28}	238.3	°C	Saturation water vapor pressure parameter for humidity

References

- [1] F. Airdi, B. De Schutter, A. Dabiri, Reinforcement learning with model predictive control for highway ramp metering, arXiv preprint, arXiv:2311.08820, 2023.
- [2] J.A.E. Andersson, J. Gillis, G. Horn, J.B. Rawlings, M. Diehl, CasADi: a software framework for nonlinear optimization and optimal control, Math. Program. Comput. 11 (2019) 1–36.
- [3] X. Blasco, M. Martínez, J.M. Herrero, C. Ramos, J. Sanchis, Model-based predictive control of greenhouse climate for reducing energy and water consumption, Comput. Electron. Agric. 55 (1) (2007) 49–70.
- [4] S. Boersma, C. Sun, S. van Mourik, Robust sample-based model predictive control of a greenhouse system with parametric uncertainty, IFAC-PapersOnLine 55 (32) (2022) 177–182.
- [5] F. Borrelli, A. Bemporad, M. Morari, Predictive Control for Linear and Hybrid Systems, Cambridge University Press, 2017.
- [6] W.H. Chen, F. You, Data-driven robust optimization for greenhouse temperature control using model predictive control, CET J., Chem. Eng. Trans. 81 (2020).
- [7] H.N. Esfahani, A.B. Kordabad, S. Gros, Approximate robust NMPC using reinforcement learning, in: 2021 European Control Conference (ECC), Netherlands, 2021, pp. 132–137.
- [8] F. García-Mañas, F. Rodríguez, M. Berenguel, J.M. Maestre, Multi-scenario model predictive control for greenhouse crop production considering market price uncertainty, IEEE Trans. Autom. Sci. Eng. 21 (3) (2024) 2936–2948.
- [9] S. Gros, M. Zanon, Data-driven economic NMPC using reinforcement learning, IEEE Trans. Autom. Control 65 (2) (2020) 636–648.
- [10] J.K. Gruber, J.L. Guzmán, F. Rodríguez, C. Bordons, M. Berenguel, J.A. Sánchez, Nonlinear MPC based on a Volterra series model for greenhouse temperature control using natural ventilation, Control Eng. Pract. 19 (4) (2011) 354–366.
- [11] M.L. Gullino, R. Albajes, P.C. Nicot (Eds.), Integrated Pest and Disease Management in Greenhouse Crops, 2nd edition, Plant Pathology in the 21st Century, vol. 9, Springer Cham, 2020.
- [12] A. Hamza, M. Ramdani, W. Bougheloum, Robust T-S fuzzy constrained predictive control design for greenhouse micro-climate, Int. J. Sci. Res. Eng. Technol. 13 (2019) 1–4.
- [13] F.L.K. Kempkes, J. Janse, S. Hemming, Greenhouse concept with high insulating double glass with coatings and new climate control strategies; from design to results from tomato experiments, in: International Symposium on New Technologies for Environment Control, Energy-Saving and Crop Production in Greenhouse and Plant 1037, 2013, pp. 83–92.
- [14] W.J.P. Kuijpers, D.J. Antunes, S. van Mourik, E.J. van Henten, M. van de Molengraft, Weather forecast error modelling and performance analysis of automatic greenhouse climate control, Biosyst. Eng. 214 (2022) 207–229.
- [15] F. Lafont, N. Pessel, J.F. Balmat, M. Fliess, On the model-free control of an experimental greenhouse, in: International Conference on Modeling, Simulation and Control, 2013.
- [16] M.G. Lagoudakis, R. Parr, M.L. Littman, Least-squares methods in reinforcement learning for control, in: Methods and Applications of Artificial Intelligence, Springer, Berlin, Heidelberg, 2002, pp. 249–260.
- [17] T.P. Lillicrap, J.J. Hunt, A. Pritzel, N. Heess, T. Erez, Y. Tassa, D. Silver, D. Wierstra, Continuous control with deep reinforcement learning, arXiv preprint, arXiv:1509.02971, 2015.
- [18] L.J. Lin, Self-improving reactive agents based on reinforcement learning, planning and teaching, Mach. Learn. 8 (3) (1992) 293–321.
- [19] F. Mahmood, R. Govindan, A. Bermak, D. Yang, T. Al-Ansari, Data-driven robust model predictive control for greenhouse temperature control and energy utilisation assessment, Appl. Energy 343 (2023) 121190.
- [20] F. Mahmood, R. Govindan, A. Bermak, D. Yang, T. Al-Ansari, Greenhouse temperature regulation in the presence of uncertainties using data-driven robust model predictive control, in: A.C. Kokossis, M.C. Georgiadis, E. Pistikopoulos (Eds.), 33rd European Symposium on Computer Aided Process Engineering, in: Computer Aided Chemical Engineering, vol. 52, Elsevier, 2023, pp. 1591–1596.
- [21] A.P. Montoya, J.L. Guzmán, F. Rodríguez, J.A. Sánchez-Molina, A hybrid-controlled approach for maintaining nocturnal greenhouse temperature: simulation study, Comput. Electron. Agric. 123 (2016) 116–124.
- [22] B. Morcego, W. Yin, S. Boersma, E.J. van Henten, V. Puig, C. Sun, Reinforcement learning versus model predictive control on greenhouse climate control, Comput. Electron. Agric. 215 (2023) 108372.
- [23] G. Schildbach, L. Fagiano, C. Frei, M. Morari, The scenario approach for stochastic model predictive control with bounds on closed-loop constraint violations, Automatica 50 (12) (2014) 3009–3018.
- [24] I. Seginer, C. Gary, M. Tchamitchian, Optimal temperature regimes for a greenhouse crop with a carbohydrate pool: a modelling study, Sci. Hortic. 60 (1–2) (1994) 55–80.
- [25] D. Silver, T. Hubert, J. Schrittwieser, L. Antonoglou, M. Lai, A. Guez, M. Lanctot, L. Sifre, D. Kumaran, T. Graepel, T. Lillicrap, K. Simonyan, D. Hassabis, A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play, Science 362 (6419) (2018) 1140–1144.
- [26] R.S. Sutton, A.G. Barto, Reinforcement Learning: An Introduction, MIT Press, 2018.

- [27] J.L. Svensen, X. Cheng, S. Boersma, C. Sun, Chance-constrained stochastic MPC of greenhouse production systems with parametric uncertainty, *Comput. Electron. Agric.* 217 (2024) 108578.
- [28] E.J. van Henten, *Greenhouse Climate Management: an Optimal Control Approach*, PhD thesis, Landbouwwuniversiteit Wageningen, Netherlands, 1994.
- [29] E.J. van Henten, Sensitivity analysis of an optimal control problem in greenhouse climate management, *Biosyst. Eng.* 85 (3) (2003) 355–364.
- [30] W. Vervaat, A relation between Brownian bridge and Brownian excursion, *Ann. Probab.* (1979) 143–149.
- [31] A. Wächter, L.T. Biegler, On the implementation of an interior-point filter line-search algorithm for large-scale nonlinear programming, *Math. Program.* 106 (2006) 25–57.
- [32] L. Wang, X. He, D. Luo, Deep reinforcement learning for greenhouse climate control, in: 2020 IEEE International Conference on Knowledge Graph (ICKG), 2020, pp. 474–480.
- [33] C.J.C.H. Watkins, P. Dayan, Q-learning, *Mach. Learn.* 8 (1992) 279–292.
- [34] D. Xu, S. Du, G. van Willigenburg, Adaptive two time-scale receding horizon optimal control for greenhouse lettuce cultivation, *Comput. Electron. Agric.* 146 (2018) 93–103.